

Trabajo 4.O

Desarrollo Colaborativo e Integración Continua mediante Git, GitHub y GitHub Actions

Antes de la redacción de este documento se aclara que, inicialmente el trabajo era colaborativo, pero gracias a una solicitud hecha al profesor se realizara la tarea por un solo estudiante, quien asume los roles tanto de revisor como de desarrollador, por lo cual a lo largo de este documento se ira aclarando que rol se ira desempeñando.

El siguiente documento, comprende en demostrar que se tiene los conocimientos sobre CI/CD, correspondiente a la materia de DevOps, por lo cual se siguen una serie de disposiciones dadas por el profesor quien imparte la materia; y para el seguimiento de las diferentes directrices el trabajo será dividido en varias fases o etapas, las cuales serán redactadas a continuación:

✓ Fase 1 — Inicialización del proyecto (Rol de desarrollador).

El proyecto en si es un mini gestor de tareas (**Task Manager**), que se basara básicamente en agregar tareas, listar tareas y marcar la tarea completada.

La elección de realizar este proyecto se detalla en los siguientes puntos:

- Más interesante que un "Hola Mundo".
- Más simple que una API completa.
- Permite agregar funcionalidades mediante ramas.
- Facilita generar conflictos de merge de forma natural.
- Encaja perfectamente con GitHub Actions.

Por lo tanto arrancamos de la siguiente manera:

```
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26
• $ touch task_manager.py

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26
• $ touch test_task_manager.py

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26
• $ touch requirements.txt

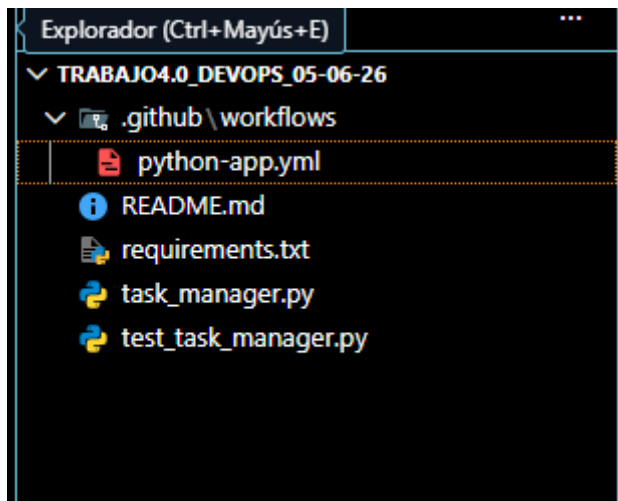
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26
• $ touch README.md

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26
• $ mkdir -p .github/workflows

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26
• $ touch .github/workflows/python-app.yml

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26
o $
```

El lenguaje que se escogió para el proyecto es Python, por lo que desde la terminal creamos todos los archivos requeridos para el proyecto además de la carpeta que permite manejar las el flujo de trabajo en Git Actions.



El directorio del proyecto debe de quedar con la estructura arquitectónica correcta.

Después a los archivos correspondientes le damos el contenido necesario para que se ejecuten sin complicaciones, por otro lado, la carpeta github/workflows quien se encarga de la comunicación con Git Actions, tiene el archivo “python-app.yml”, dicho archivo contiene toda la configuración de instalación de dependencias, correcta comunicación entre el código y Git Actions, el archivo en cuestión quedaría como se muestra en la siguiente imagen:



Después de esto vamos a hacer las pruebas locales para poder verificar su correcto funcionamiento antes de la conexión con un repositorio de GitHub.

```

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26
$ pip install -r requirements.txt
Requirement already satisfied: pytest in c:\users\pc\appdata\local\programs\python\python313\lib\site-packages (from -r requirements.txt (line 1)) (8.4.2)
Requirement already satisfied: colorama>=0.4 in c:\users\pc\appdata\local\programs\python\python313\lib\site-packages (from pytest->-r requirements.txt (line 1)) (0.4.6)
Requirement already satisfied: iniconfig>=1 in c:\users\pc\appdata\local\programs\python\python313\lib\site-packages (from pytest->-r requirements.txt (line 1)) (2.1.0)
Requirement already satisfied: packaging>=20 in c:\users\pc\appdata\local\programs\python\python313\lib\site-packages (from pytest->-r requirements.txt (line 1)) (25.0)
Requirement already satisfied: pluggy<2,>=1.5 in c:\users\pc\appdata\local\programs\python\python313\lib\site-packages (from pytest->-r requirements.txt (line 1)) (1.6.0)
Requirement already satisfied: pygments>=2.7.2 in c:\users\pc\appdata\local\programs\python\python313\lib\site-packages (from pytest->-r requirements.txt (line 1)) (2.19.2)

[notice] A new release of pip is available: 24.3.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip

```

Comenzamos con la instalación de “pytest”.

```

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26
$ python task_manager.py
['Estudiar DevOps', 'Practicar GitHub']

```

Después de la instalación realizamos una prueba local, con esto se verifica su funcionamiento.

```

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26
$ pytest
===== test session starts =====
platform win32 -- Python 3.13.2, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\PC\Desktop\Yavirac Pablo Reyes\Fifth\2 oportunidad\DevOps\trabajo4.0_devops_05-06-26
collected 1 item

test_task_manager.py . [100%]

===== 1 passed in 0.12s =====

```

Con esto hemos verificado su estado funcional.

✓ Fase 2 — Creación del repositorio remoto (Rol de revisor).

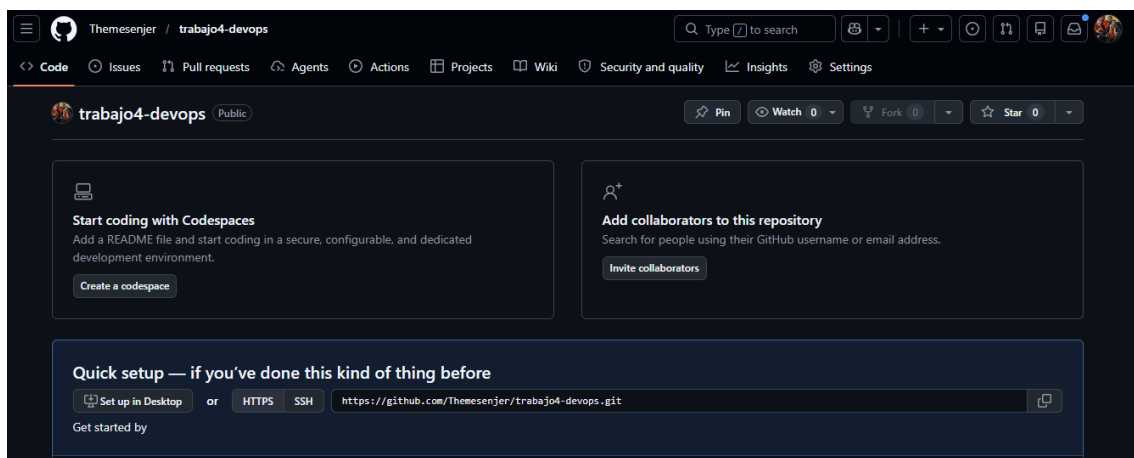
```

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26
$ git init
Initialized empty Git repository in C:/Users/PC/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26/.git/

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (master)
$ git branch -M main

```

La inicialización de git, y el cambio de rama es lo primero que ejecutaremos, para después proseguir a la creación del repositorio directamente en GitHub.



Tenemos la creación del repositorio.

```
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (main)
$ git remote add origin https://github.com/Themesenjer/trabajo4-devops.git
```

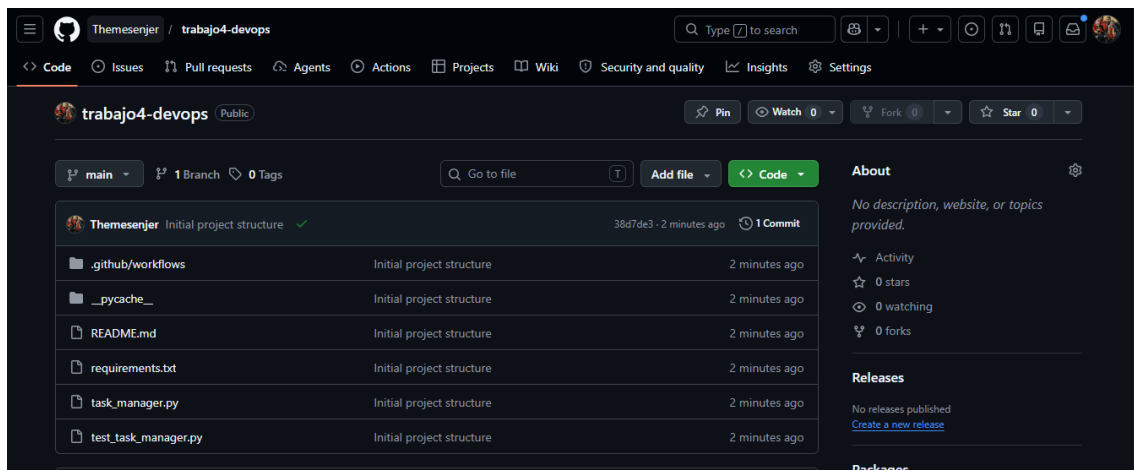
Lo conectamos con el proyecto.

```
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (main)
$ git add .
warning: in the working copy of '.github/workflows/python-app.yml', CRLF will be replaced by LF the next time Git touches it
warning: in the working copy of 'task_manager.py', CRLF will be replaced by LF the next time Git touches it
warning: in the working copy of 'test_task_manager.py', CRLF will be replaced by LF the next time Git touches it

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (main)
$ git commit -m "Initial project structure"
[main (root-commit) 38d7de3] Initial project structure
7 files changed, 64 insertions(+)
create mode 100644 .github/workflows/python-app.yml
create mode 100644 README.md
create mode 100644 __pycache__/task_manager.cpython-313.pyc
create mode 100644 __pycache__/test_task_manager.cpython-313-pytest-8.4.2.pyc
create mode 100644 requirements.txt
create mode 100644 task_manager.py
create mode 100644 test_task_manager.py

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (main)
$ git push -u origin main
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 8 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (12/12), 2.78 KiB | 189.00 KiB/s, done.
Total 12 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Themesenjer/trabajo4-devops.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'
```

Realizamos el primer commit.



Al refrescar el navegador, podemos observar la correcta conexión al visualizar la estructura del proyecto en el repositorio.

✓ Fase 3 — Primera rama (Rol de desarrollador).

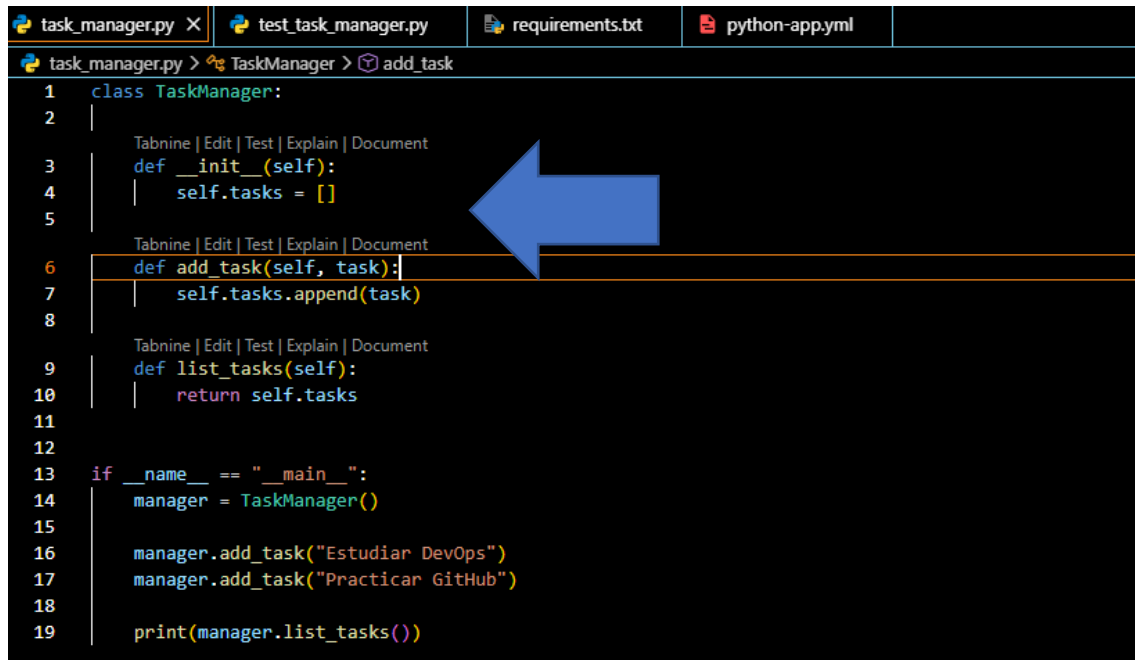
Creación de la rama.

```
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (main)
$ git checkout -b feature/add-complete-task
Switched to a new branch 'feature/add-complete-task'

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (feature/add-complete-task)
$ git branch
* feature/add-complete-task
main
```

Esto demuestra la creación de la rama de desarrollo y su verificación de estar en la rama correcta.

Ahora en el rol de desarrollador vamos a agregar una nueva funcionalidad, o en otras palabras vamos a mejorar el gestor de tareas.



```
task_manager.py X test_task_manager.py requirements.txt python-app.yml
task_manager.py > TaskManager > add_task

1 class TaskManager:
2 |
3 |   Tabnine | Edit | Test | Explain | Document
4 |   def __init__(self):
5 |       self.tasks = []
6 |
7 |   Tabnine | Edit | Test | Explain | Document
8 |   def add_task(self, task):
9 |       self.tasks.append(task)
10 |
11 |   Tabnine | Edit | Test | Explain | Document
12 |   def list_tasks(self):
13 |       return self.tasks
14 |
15 |
16 | if __name__ == "__main__":
17 |     manager = TaskManager()
18 |
19 |     manager.add_task("Estudiar DevOps")
20 |     manager.add_task("Practicar GitHub")
21 |
22 |     print(manager.list_tasks())
```

Actualmente tenemos el guardado automático de esta manera.



```
task_manager.py X test_task_manager.py requirements.txt python-app.yml
task_manager.py > TaskManager

1 class TaskManager:
2 |
3 |   Tabnine | Edit | Test | Explain | Document
4 |   def __init__(self):
5 |       self.tasks = []
6 |
7 |   Tabnine | Edit | Test | Explain | Document
8 |   def add_task(self, task):
9 |       self.tasks.append({
10 |           "name": task,
11 |           "completed": False
12 |       })
13 |
14 |   Tabnine | Edit | Test | Explain | Document
15 |   def complete_task(self, index):
16 |       self.tasks[index]["completed"] = True
17 |
18 |   Tabnine | Edit | Test | Explain | Document
19 |   def list_tasks(self):
20 |       return self.tasks
21 |
22 |
23 | if __name__ == "__main__":
24 |     manager = TaskManager()
25 |
26 |     manager.add_task("Estudiar DevOps")
27 |     manager.add_task("Practicar GitHub")
28 |
29 |     manager.complete_task(0)
30 |
31 |     print(manager.list_tasks())
```

La actualización mejora significativamente el guardado.

Adicional a esto tambien actualizamos el módulo de pruebas (test_task_manager.py).

Y realizamos las pruebas locales correspondientes.

```
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (feature/add-complete-task)
$ python task_manager.py
[{'name': 'Estudiar DevOps', 'completed': True}, {'name': 'Practicar GitHub', 'completed': False}]

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (feature/add-complete-task)
$ pytest
===== test session starts =====
platform win32 -- Python 3.13.2, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\PC\Desktop\Yavirac Pablo Reyes\Fifth\2 oportunidad\DevOps\trabajo4.0_devops_05-06-26
collected 2 items

test_task_manager.py .. [100%]

===== 2 passed in 0.14s =====
```

Si las pruebas pasan sin problemas después de las modificaciones, estamos listos para realizar un nuevo commit.

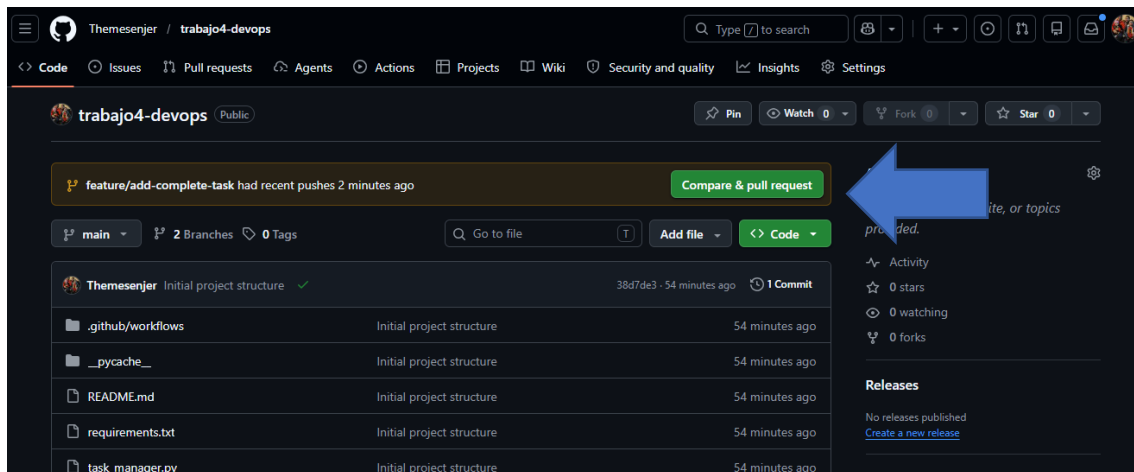
```
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (feature/add-complete-task)
$ git add .
warning: in the working copy of 'task_manager.py', CRLF will be replaced by LF the next time Git touches it
warning: in the working copy of 'test_task_manager.py', CRLF will be replaced by LF the next time Git touches it

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (feature/add-complete-task)
$ git commit -m "Add task completion feature"
[feature/add-complete-task b06581a] Add task completion feature
4 files changed, 24 insertions(+), 2 deletions(-)
```

Este es el primer commit de la nueva rama, commit hecho con el rol de desarrollador.

```
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (feature/add-complete-task)
$ git push origin feature/add-complete-task
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 1.74 KiB | 104.00 KiB/s, done.
Total 7 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
remote:
remote: Create a pull request for 'feature/add-complete-task' on GitHub by visiting:
remote:   https://github.com/Themesenjer/trabajo4-devops/pull/new/feature/add-complete-task
remote:
To https://github.com/Themesenjer/trabajo4-devops.git
 * [new branch]   feature/add-complete-task -> feature/add-complete-task
```

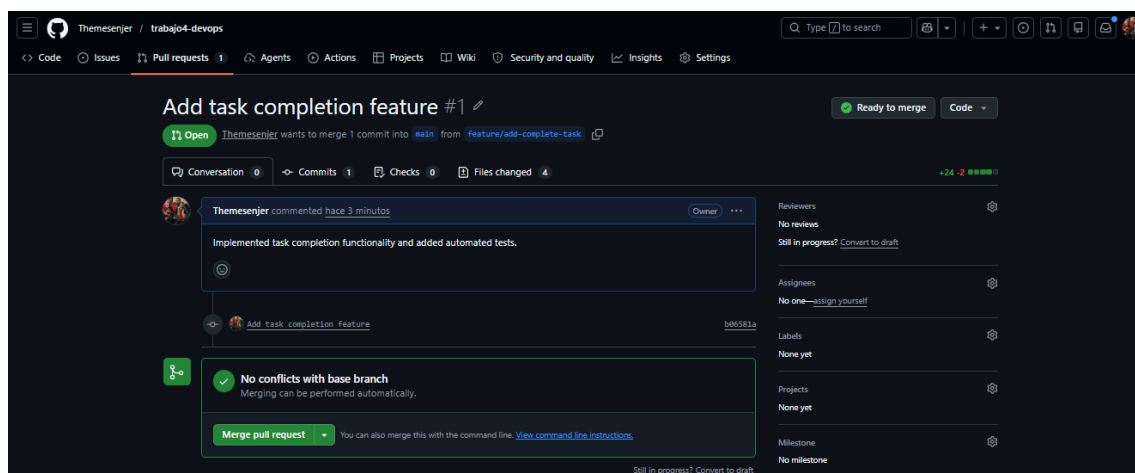
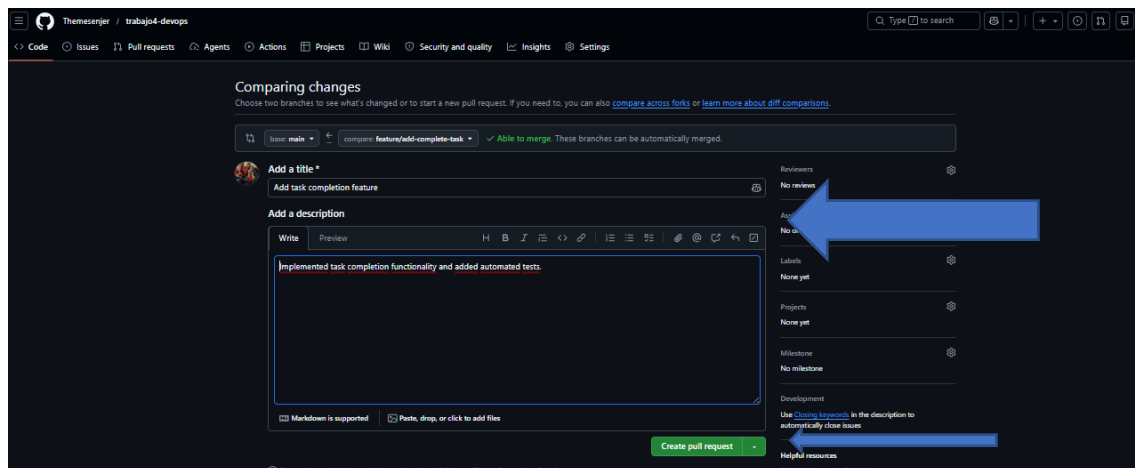
Publicación de los cambios al repositorio remoto actual.



Verificamos los nuevos cambios en GitHub en donde debe aparecer la nueva rama creada.

✓ Fase 4 — Pull Request (Rol de desarrollador).

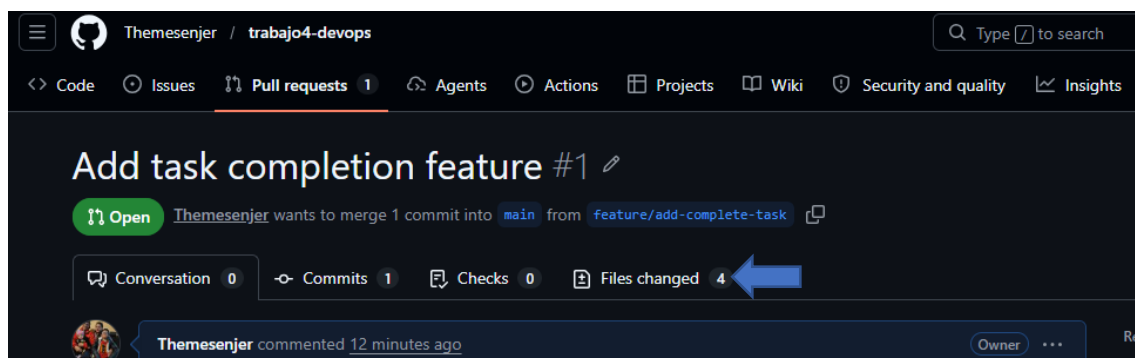
Después del push GitHub nos muestra: “Compare & pull request”, hacemos click ahí y le damos un título y una descripción, y damos click en “Create pull request”.

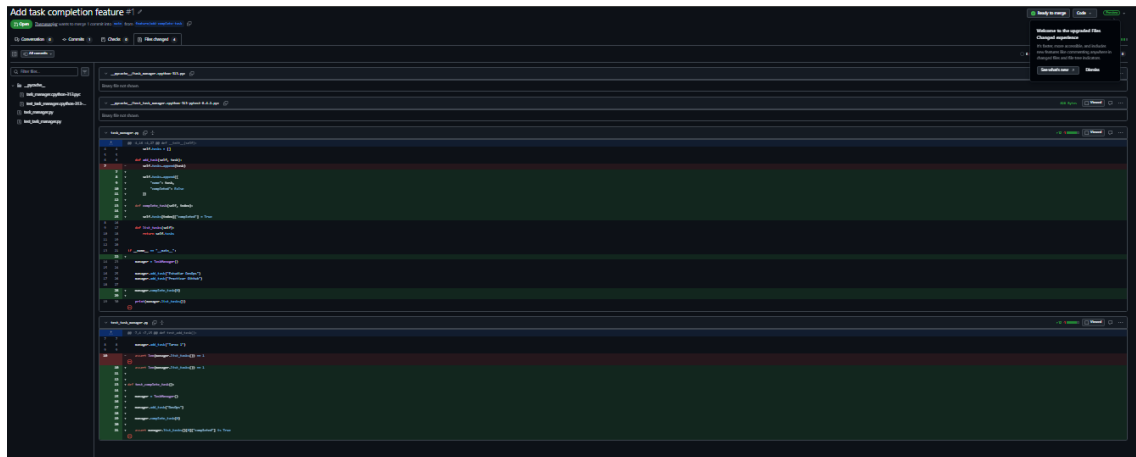


Creación del Pull Request

✓ Fase 5 - Revisión del Pull Request (Rol de revisor).

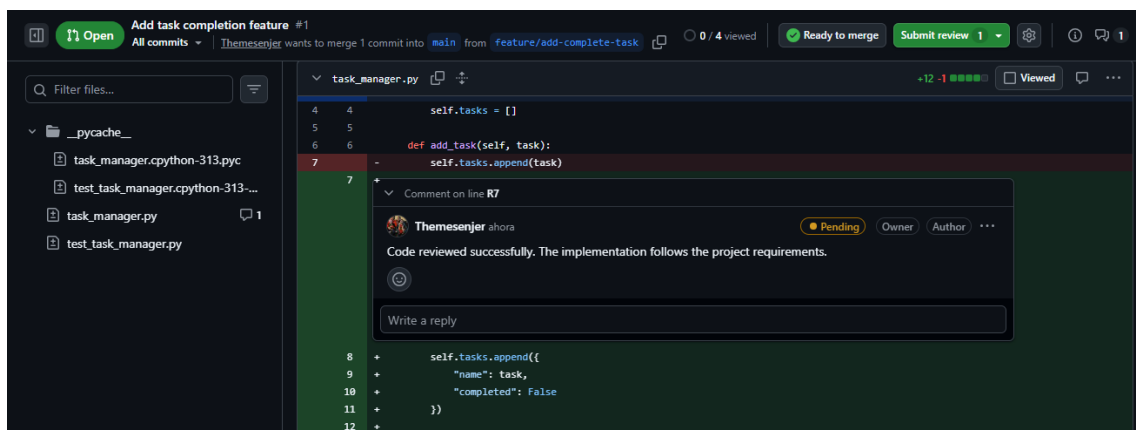
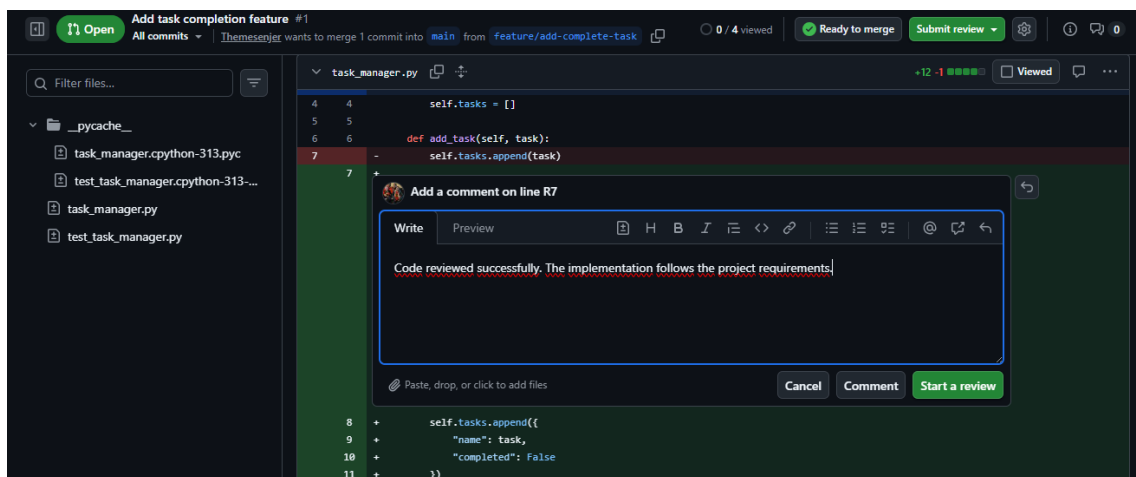
Con el Pull Request creado y abierto, nos dirigimos a la pestaña “Files changed”, y así GitHub nos mostrara las líneas modificadas.





Revisión de código del Pull Request.

Agregamos un comentario de revisión, en cualquier línea modificada.



Comentario de revisión realizado.

✓ Fase 6 - Aprobar el Pull Request (Rol de revisor).

En la parte superior derecha del PR, vemos “Review changes” seleccionamos “Aprove”, Antes de seguir, desde aquí vamos a realizar algo más, lo que se va a hacer es usar otra cuenta de GitHub, esto se debe a que la cuenta actual no permite autorizar los cambios,

es decir GitHub muchas veces oculta la opción "Review changes" porque detecta somos el mismo autor del PR y el mismo propietario del repositorio.

¿Entonces para qué sirve la segunda cuenta?

La segunda cuenta la vamos a usar para las siguientes evidencias:

- Aprobación real de Pull Request
- Merge real entre usuarios distintos
- Conflicto de integración
- Resolución del conflicto

En otras palabras, las siguientes actividades las aremos con los dos usuarios, por lo cual vamos a seguir una serie de pasos detallados a continuación:

- Clonar el repositorio actual

```
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps
$ mkdir colaborador-devops

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps
$ cd colaborador-devops

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/colaborador-devops
$ git clone https://github.com/Themesenjer/trabajo4-devops.git
Cloning into 'trabajo4-devops'...
remote: Enumerating objects: 19, done.
remote: Counting objects: 100% (19/19), done.
remote: Compressing objects: 100% (11/11), done.
remote: Total 19 (delta 5), reused 18 (delta 4), pack-reused 0 (from 0)
Receiving objects: 100% (19/19), 4.32 KiB | 232.00 KiB/s, done.
Resolving deltas: 100% (5/5), done.
```

En nuestro entorno local creamos una nueva carpeta, y nos clonamos el repositorio.

- Configurar Git

```
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/colaborador-devops
$ cd trabajo4-devops

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/colaborador-devops/trabajo4-devops (main)
$ git config user.name "pabloreyes093"

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/colaborador-devops/trabajo4-devops (main)
$ git config user.email "pablo.jrr.1993@gmail.com"
```

Con esto configuramos Git para esta carpeta, usando las credenciales de la otra cuenta de Git.

- Verificación

```

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/colaborador-devops/trabajo4-devops (main)
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
http.sslcainfo=C:/Program Files/Git/mingw64/etc/ssl/certs/ca-bundle.crt
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
core.editor="C:\Users\PC\AppData\Local\Programs\Microsoft VS Code\bin\code" --wait
it
core.autocrlf=input
user.name=Pablo Reyes
user.email=pjr.reyes@yavirac.edu.ec
credential.helper=store

```

Con este comando abrimos un listado largo dentro de un paginador.

Al final del listado debe de aparecer las credenciales de la segunda cuenta de Git, y con eso Git usara esa cuenta para los commits que hagan dentro de esta copia del proyecto.

Si necesitamos estar mas seguros hacemos lo siguiente:

```

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/colaborador-devops/trabajo4-devops (main)
$ git config user.name
pabloryes093

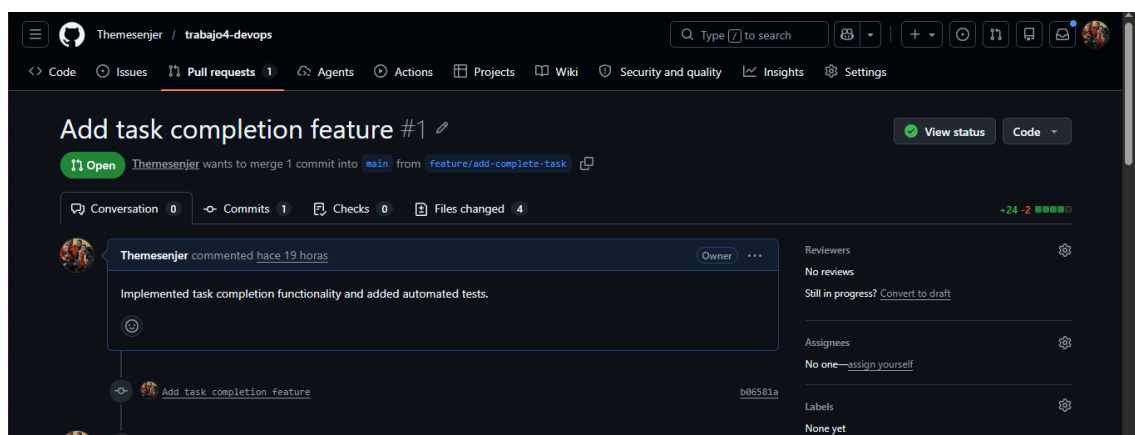
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/colaborador-devops/trabajo4-devops (main)
$ git config user.email
pablo.jrr.1993@gmail.com

```

Con esto verificamos las credenciales.

Ahora lo siguiente es saber en que parte estamos hasta aquí con todas las modificaciones hechas.

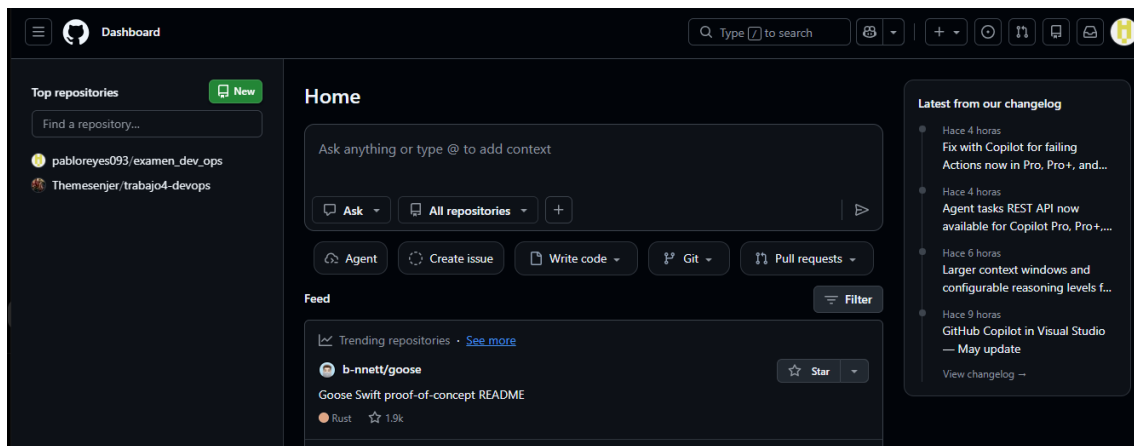
En la cuenta principal de Git tenemos esto:



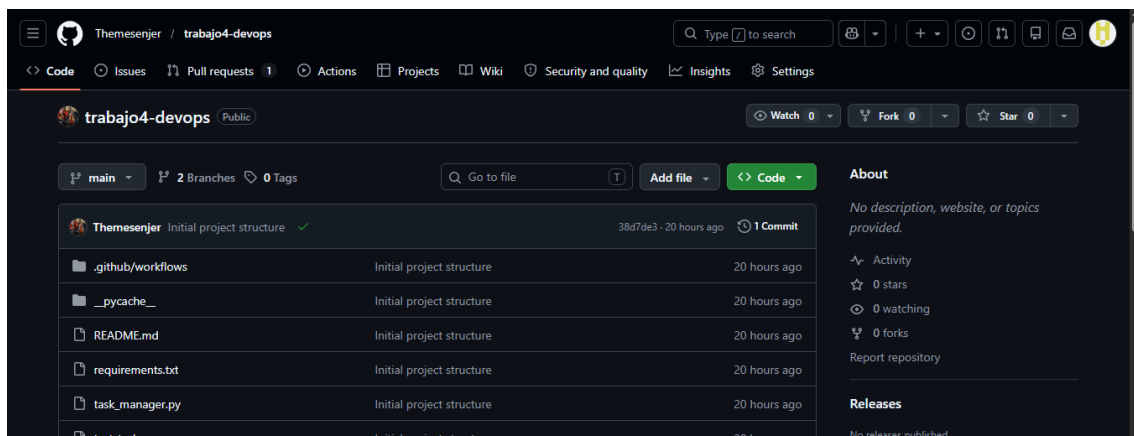
Por lo que nos dice que nuestro PR esta en estado “OPEN”, además de que esta imagen muestra algo muy importante: “Themesenjer started a review Pending”, lo que significa que comenzamos una revisión, pero no la terminamos.

Entonces desde este punto, como ya tenemos la segunda cuenta preparada, vamos a aprovecharla para que la aprobación sea real entre dos usuarios. Por lo que vamos a tener el PR abierto sin hacer ninguna modificación.

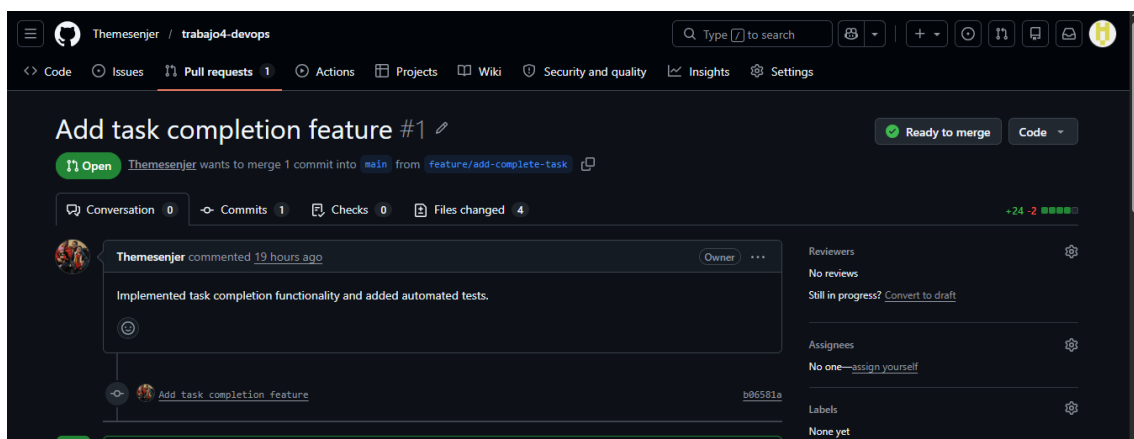
El siguiente paso es dirigirnos a la segunda cuenta de Git.



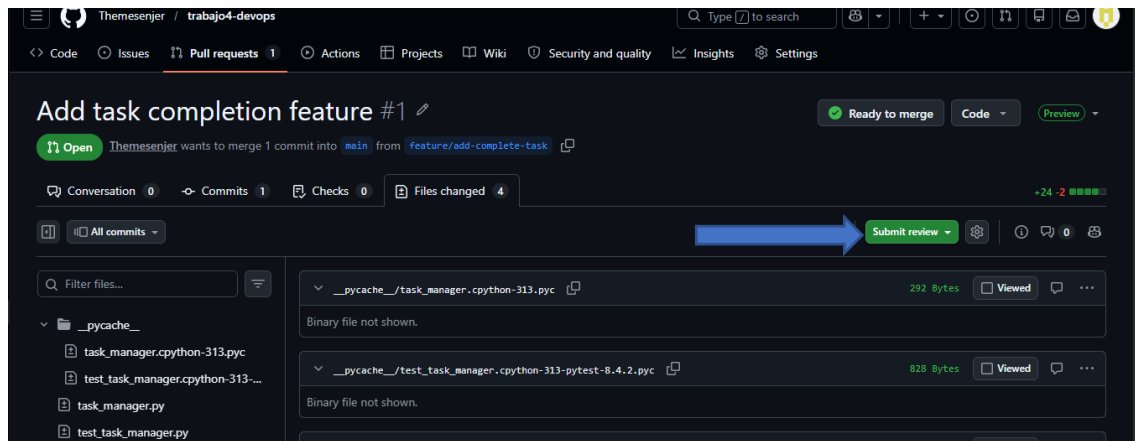
Estamos en la segunda cuenta de Git.



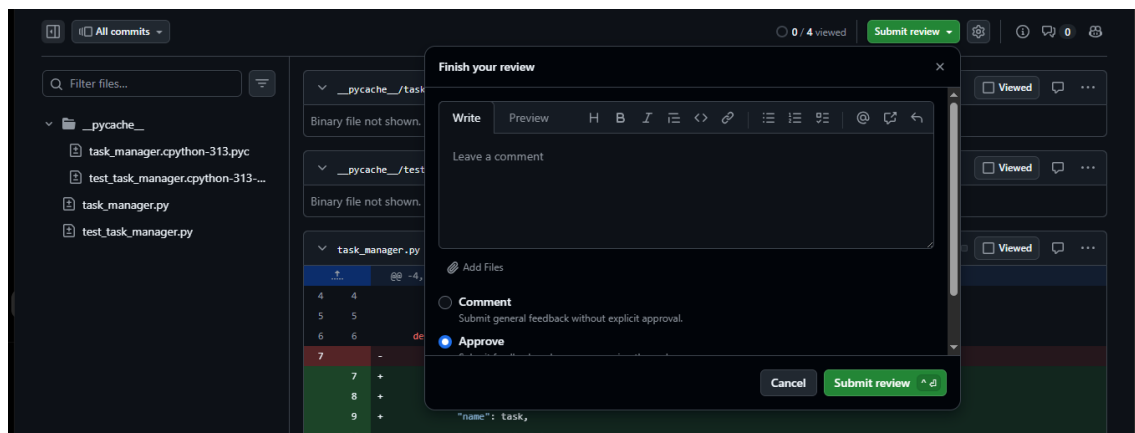
Estamos en el repositorio principal, “Themesenjer/trabajo4-devops”.



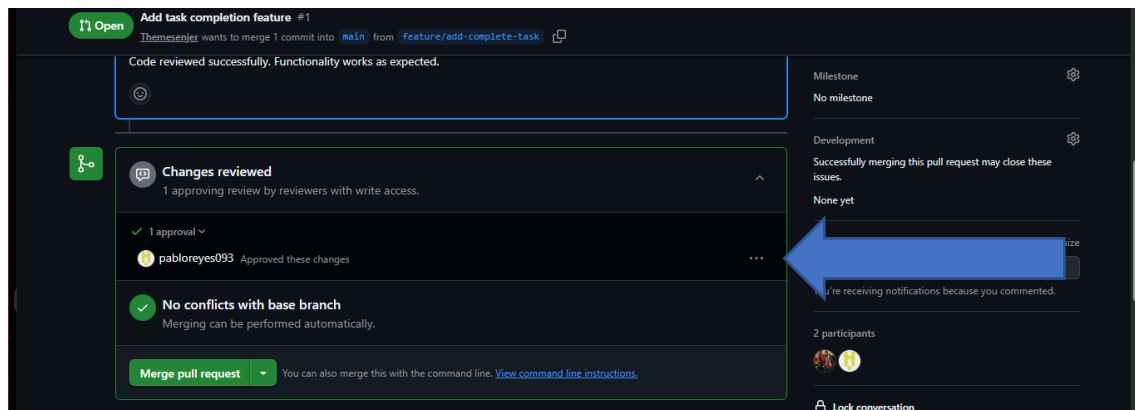
Vamos al Pull Request #1.



Hacemos click en “Submit review”.

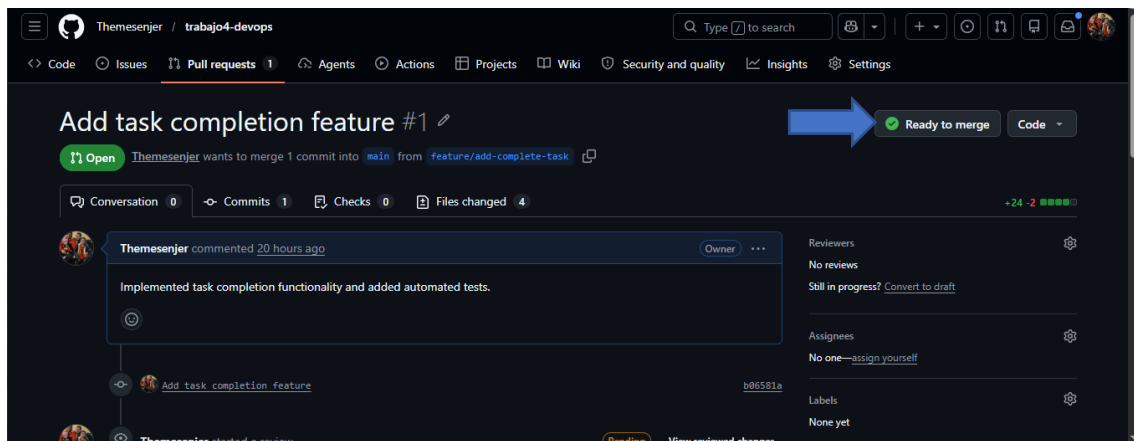


Seleccionamos “Aprove” y le damos un comentario sencillo, y finalizamos presionando “Submit review”.

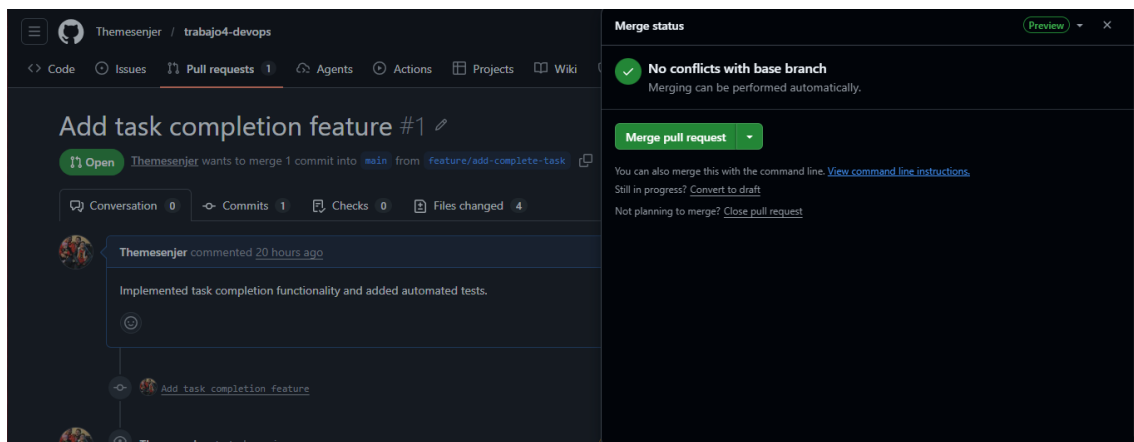


Al haber presionado “Submit review” aparecerá la aprobación de la revisión del código por parte del colaborador, que en este caso es la segunda cuenta usada de Git. En otras palabras, el rol de revisor se esta ejecutando con las métricas solicitadas.

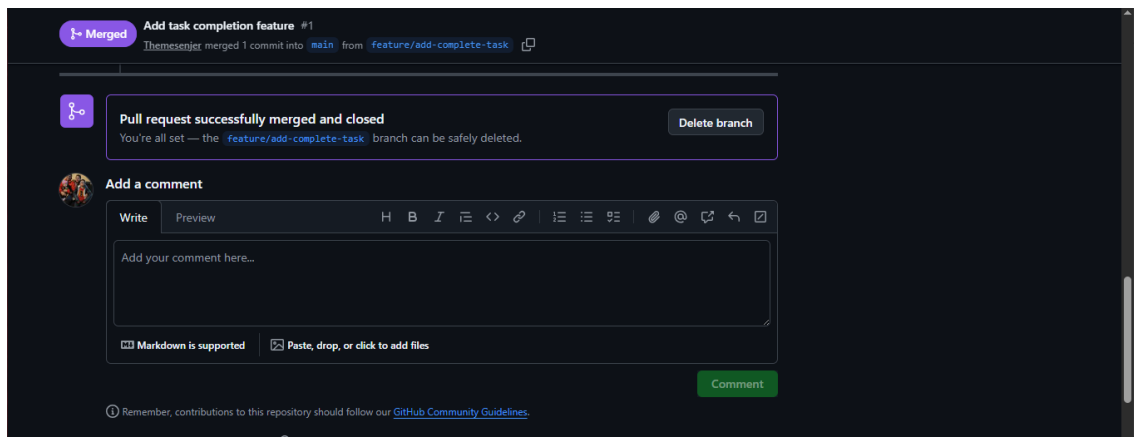
Ahora volvemos a la cuenta principal de Git y en el PR debe de aparecer el botón “Merge pull request”, en donde vamos a hacer el Merge.



Presionamos el botón “Ready to Merge”.



Después presionamos el botón Merge Pull Request, y hacemos el Merge.



Revisión y aprobación del Pull Request.

¿Qué se hizo hasta ahora?

El colaborador pabloryes093 (segunda cuenta de Git) revisó los cambios implementados en la rama “feature/add-complete-task” y verificó que la funcionalidad cumplía los requisitos del proyecto.

Posteriormente se aprobó el Pull Request mediante la opción “Approve”, agregando un comentario de validación del código.

- Integración de cambios (Merge)

Una vez aprobados los cambios, el propietario del repositorio realizó el Merge del Pull Request hacia la rama principal (main), integrando exitosamente la nueva funcionalidad al proyecto.

✓ Fase 7 – Generación y resolución de conflictos de integración (2 roles se ejecutan al unisonó).

Esta fase tiene como objetivo provocar intencionalmente un conflicto entre dos ramas para luego resolverlo y documentarlo.

1. Crear la rama conflicto-a

Desde el repositorio principal ejecutamos lo siguiente:

```
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (feature/add-complete-task)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
(.venv)
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (main)
$ git pull origin main
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 922 bytes | 35.00 KiB/s, done.
From https://github.com/Themesenjer/trabajo4-devops
* branch      main      -> FETCH HEAD
  38d7de3..707bfb7 main    -> origin/main
Updating 38d7de3..707bfb7
Fast-forward
 _pycache_/task_manager.cpython-313.pyc | Bin 1231 -> 1523 bytes
 .../test_task_manager.cpython-313-pytest-8.4.2.pyc | Bin 1916 -> 2744 bytes
 task_manager.py | 13 ++++++++
 test_task_manager.py | 13 ++++++++
 4 files changed, 24 insertions(+), 2 deletions(-)
(.venv)
```

Nos cambiamos de rama y actualizamos la rama, pero aquí se observa algo interesante “Updating 38d7de3..707bfb7” “Fast-forward”, eso significa que main ya recibió correctamente el merge anterior (feature/add-complete-task) antes de crear conflicto-a.

```
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (main)
$ git checkout -b conflicto-a
Switched to a new branch 'conflicto-a'
(.venv)
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (conflicto-a)
$ git branch
* conflicto-a
  feature/add-complete-task
  main
(.venv)
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (conflicto-a)
```

Creamos una nueva rama y verificamos.

2. Modificar task_manager.py

```
task_manager.py X test_task_manager.py requirements.txt python-app.yml
task_manager.py > ...
1 class TaskManager:
12
    Tabnine | Edit | Test | Explain | Document
13 def complete_task(self, index):
14
    self.tasks[index]["completed"] = True
15
16
    Tabnine | Edit | Test | Explain | Document
17 def list_tasks(self):
18
    return self.tasks
19
20
21 if __name__ == "__main__":
22
    manager = TaskManager()
23
24
    manager.add_task("Estudiar DevOps")
25
    manager.add_task("Practicar GitHub")
26
27
    manager.complete_task(0)
28
29
30 print(manager.list_tasks())
```

```
task_manager.py M X test_task_manager.py requirements.txt
task_manager.py > ...
1 class TaskManager:
13 def complete_task(self, index):
15
    self.tasks[index]["completed"] = True
16
17
    Tabnine | Edit | Test | Explain | Document
18 def list_tasks(self):
19
    return self.tasks
20
21 if __name__ == "__main__":
22
    manager = TaskManager()
23
24
    manager.add_task("Estudiar DevOps")
25
    manager.add_task("Practicar GitHub - Version A")
26
27
28
29
30 print(manager.list_tasks())
```

Realizamos un cambio controlado en el código.

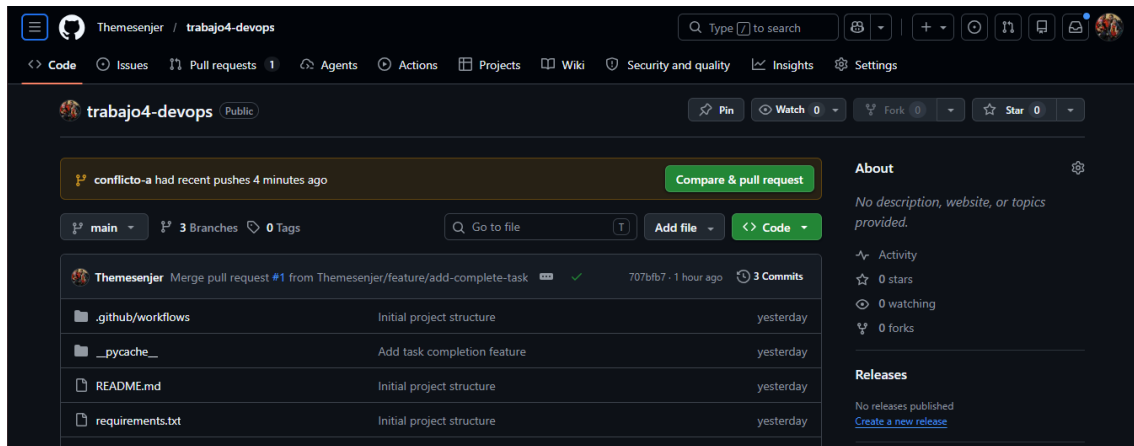
3. Commit y Push

```
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (conflicto-a)
$ git add .
(.venv)
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (conflicto-a)
$ git commit -m "Add Version A task"
[conflicto-a 70a8334] Add Version A task
1 file changed, 1 insertion(+), 1 deletion(-)
(.venv)
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/trabajo4.0_devops_05-06-26 (conflicto-a)
$ git push -u origin conflicto-a
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 312 bytes | 104.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'conflicto-a' on GitHub by visiting:
remote:   https://github.com/Themesenjer/trabajo4-devops/pull/new/conflicto-a
remote:
To https://github.com/Themesenjer/trabajo4-devops.git
 * [new branch]      conflicto-a -> conflicto-a
branch 'conflicto-a' set up to track 'origin/conflicto-a'.
Q (.venv)
```

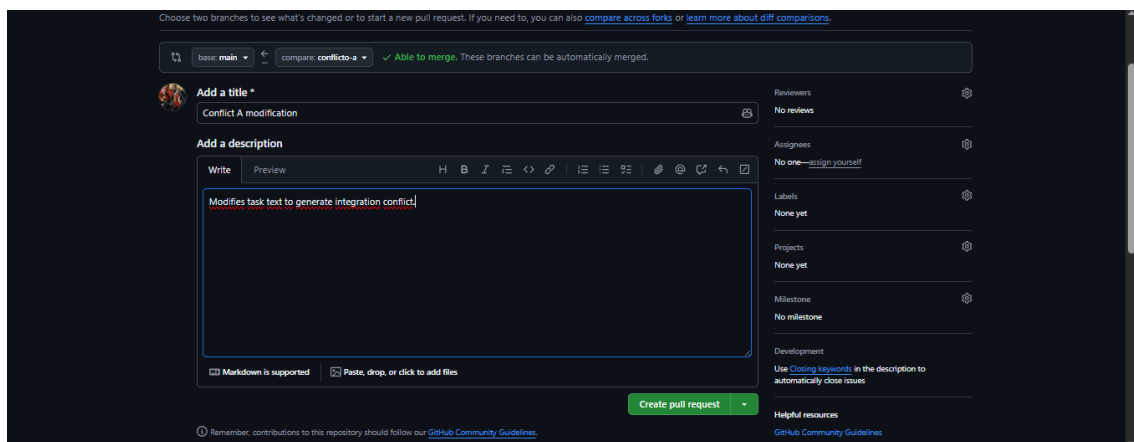
Realizamos un commit y empujamos los cambios.

4. Crear Pull Request

Ahora vamos al Git principal, en donde aparecerá la nueva rama y la opción para crear el PR.



Creamos el nuevo PR.



Le damos un título y una descripción , finalizamos con la creación.

Lo importante de aquí es, no crear el Merge ni aprobarlo, más bien la continuación es irnos al repositorio secundario o clonado.

```
PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/colaborador-devops/trabajo4-devops (main)
$ git branch
* main

PC@DESKTOP-0TN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunidad/DevOps/colaborador-devops/trabajo4-devops (main)
$ git pull origin main
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 922 bytes | 115.00 KiB/s, done.
From https://github.com/Themesenjer/trabajo4-devops
* branch          main          -> FETCH_HEAD
   38d7de3..707bfb7  main        -> origin/main
Updating 38d7de3..707bfb7
Fast-forward
 __pycache__/task_manager.cpython-313.pyc | Bin 1231 -> 1523 bytes
 .../test_task_manager.cpython-313-pytest-8.4.2.pyc | Bin 1916 -> 2744 bytes
 task_manager.py | 13 ++++++++
 test_task_manager.py | 13 ++++++++
 4 files changed, 24 insertions(+), 2 deletions(-)
```

Sincronizamos con GitHub.


```
PC@DESKTOP-OTN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunity/DevOps/colaborador-devops/trabajo4-devops (main)
$ git checkout -b conflict-b
Switched to a new branch 'conflict-b'

PC@DESKTOP-OTN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunity/DevOps/colaborador-devops/trabajo4-devops (conflict-b)
$ git branch
* conflict-b
  main
```

Creamos la rama y verificamos.

Ahora modificaremos la misma línea de código, como en el anterior paso.

```
MINGW64:/c/Users/PC/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunity/DevOps/colaborador-devops/trabajo4-devops
GNU nano 8.0 task_manager.py
class TaskManager:

    def __init__(self):
        self.tasks = []

    def add_task(self, task):

        self.tasks.append({
            "name": task,
            "completed": False
        })

    def complete_task(self, index):

        self.tasks[index]["completed"] = True

    def list_tasks(self):
        return self.tasks

if __name__ == "__main__":

    manager = TaskManager()

    manager.add_task("Estudiar DevOps")
    manager.add_task("Practicar GitHub")

    manager.complete_task(0)
```

```
MINGW64:/c/Users/PC/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunity/
GNU nano 8.0
class TaskManager:

    def __init__(self):
        self.tasks = []

    def add_task(self, task):

        self.tasks.append({
            "name": task,
            "completed": False
        })

    def complete_task(self, index):

        self.tasks[index]["completed"] = True

    def list_tasks(self):
        return self.tasks

if __name__ == "__main__":

    manager = TaskManager()

    manager.add_task("Estudiar DevOps")
    manager.add_task("Practicar GitHub - Version B")

    manager.complete_task(0)
```

Líneas de código modificadas.

```
PC@DESKTOP-OTN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunity/DevOps/colaborador-devops/trabajo4-devops (conflict-b)
$ git add .

PC@DESKTOP-OTN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunity/DevOps/colaborador-devops/trabajo4-devops (conflict-b)
$ git commit -m "Modify task for conflict B"
[conflict-b 60f37ad] Modify task for conflict B
1 file changed, 2 insertions(+), 2 deletions(-)

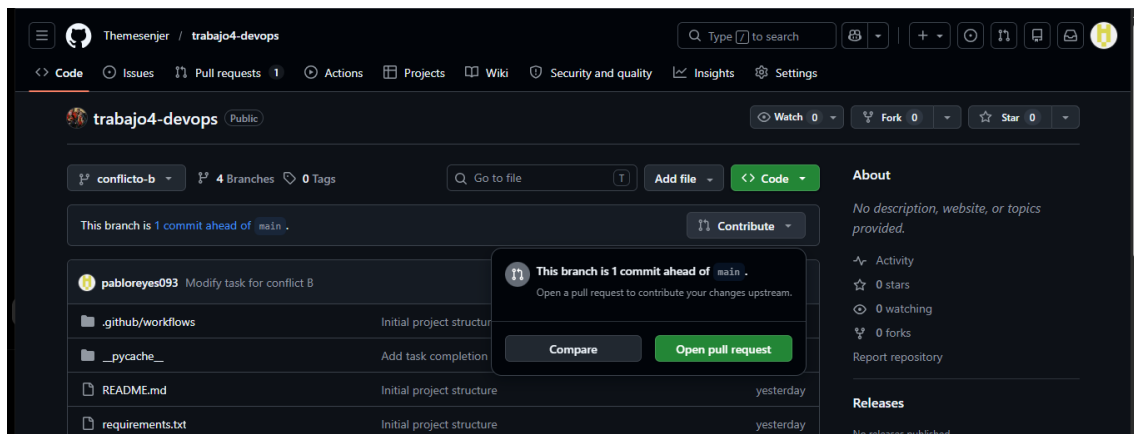
PC@DESKTOP-OTN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunity/DevOps/colaborador-devops/trabajo4-devops (conflict-b)
$ git push -u conflict-b
fatal: 'conflict-b' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.

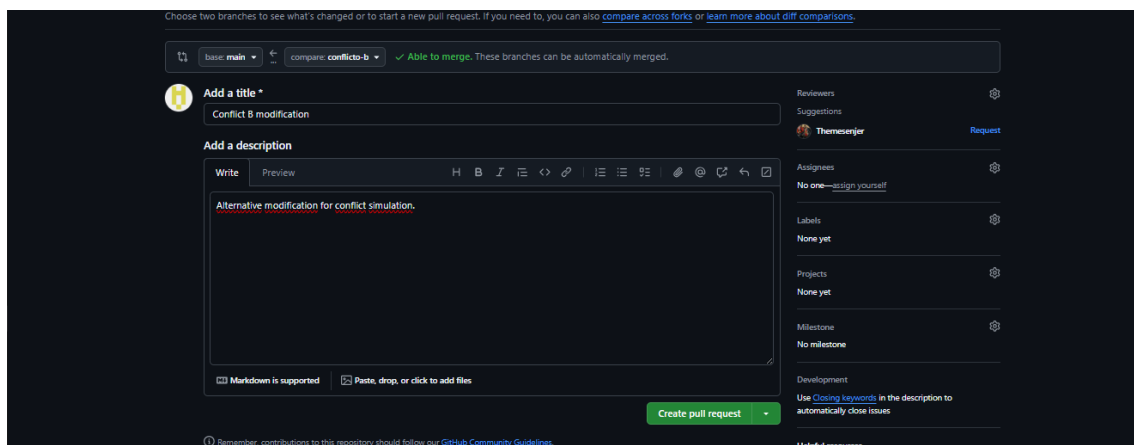
PC@DESKTOP-OTN2VCT MINGW64 ~/Desktop/Yavirac Pablo Reyes/Fifth/2 oportunity/DevOps/colaborador-devops/trabajo4-devops (conflict-b)
$ git push -u origin conflict-b
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 322 bytes | 80.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'conflict-b' on GitHub by visiting:
remote:   https://github.com/Themesenjer/trabajo4-devops/pull/new/conflict-b
remote:
To https://github.com/Themesenjer/trabajo4-devops.git
 * [new branch]      conflict-b -> conflict-b
branch 'conflict-b' set up to track 'origin/conflict-b'.
```

Hacemos un commit y empujamos los cambios.

Ahora el siguiente paso es crear el PR desde GitHub.



Buscamos la rama creada en el repositorio secundario y creamos el PR.

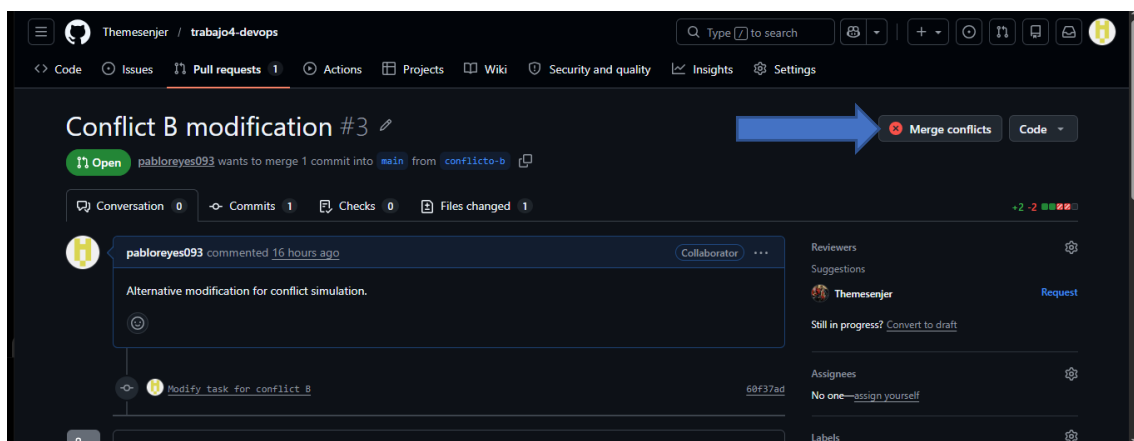


Le damos un título y una descripción, y lo creamos.

Lo importante de aquí es que cuando el PR este creado, no hagamos un Merge.

5. Resolución de conflictos

Lo que vamos a tratar de hacer es un error controlado para así resolverlo, con el fin de conocer cómo resolver conflictos.



Como podemos observar ahora tenemos un conflicto, y eso es justamente lo que buscábamos o se quería provocar.

¿Por qué el error?

Cuando GitHub muestra "Merge conflicts", significa que:

- La rama conflicto-a modificó una línea.
- La rama conflicto-b modificó la misma línea.
- Git no sabe cuál de las dos versiones conservar.
- Se requiere intervención manual.

Y eso cumple con uno de los requisitos más importantes del trabajo:

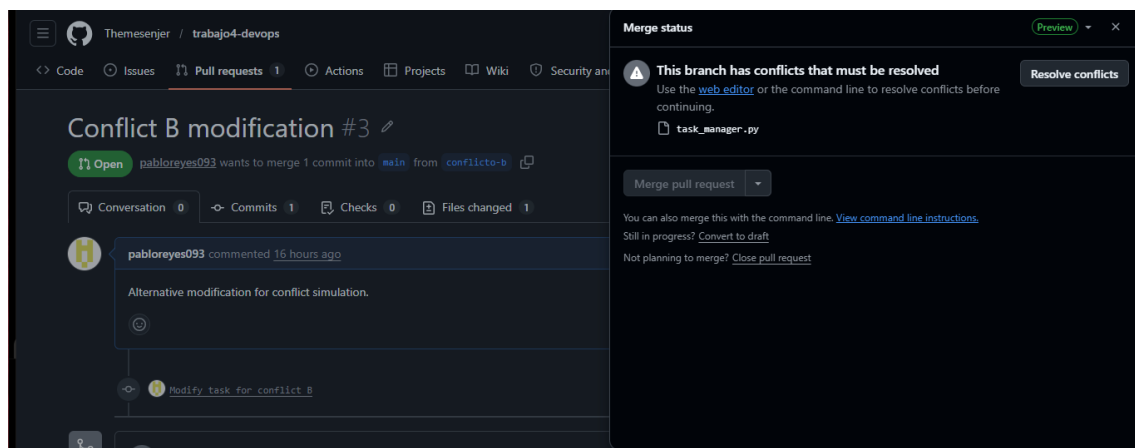
- Generación de conflictos de integración
- Resolución de conflictos de integración

Ahora es necesario entender en que punto estamos.

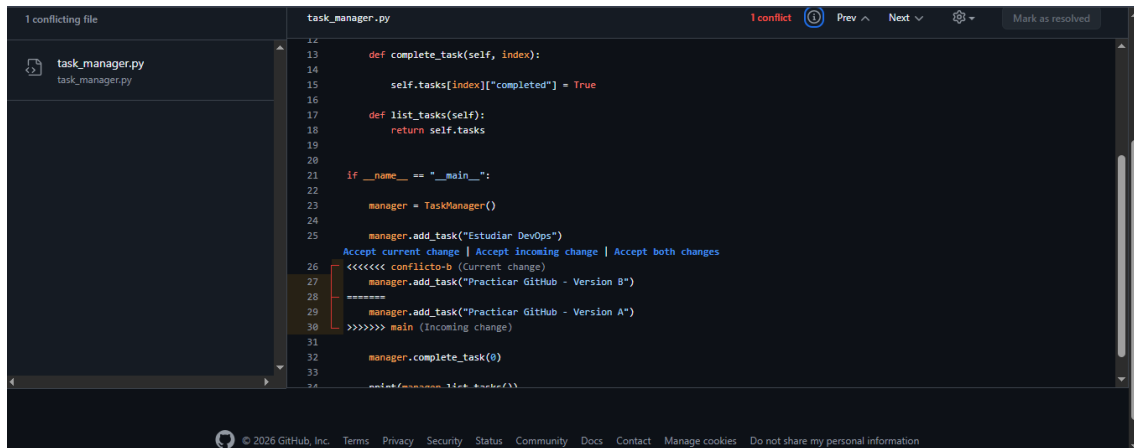
Hasta ahora tenemos:

Actividad	Estado
Crear repositorio	Ok
Crear rama	Ok
Commits	Ok
Push remoto	Ok
Pull Request	Ok
Revisión de código	Ok
Aprobación de PR	Ok
Merge	Ok
Simulación de segundo colaborador	Ok
Generación de conflicto	Ok

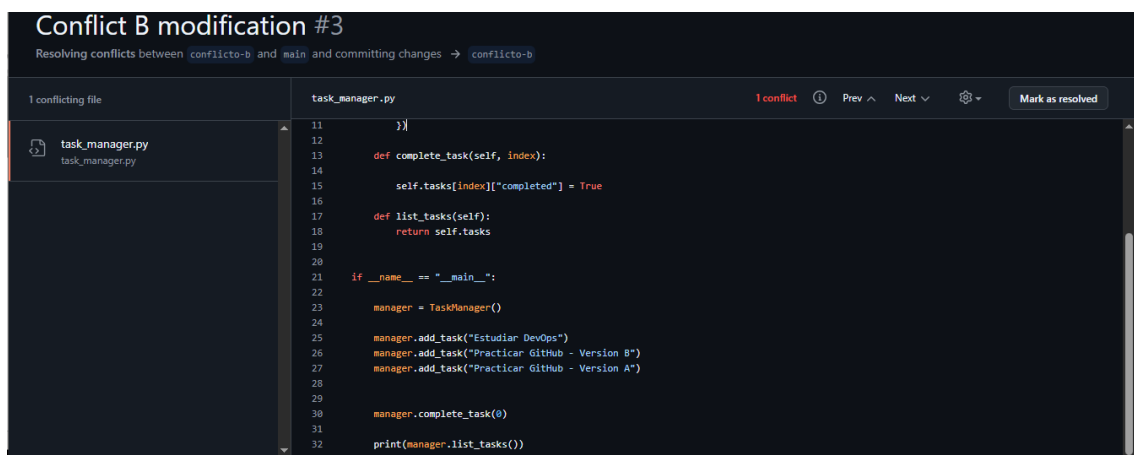
Ahora lo que falta es resolver el conflicto.



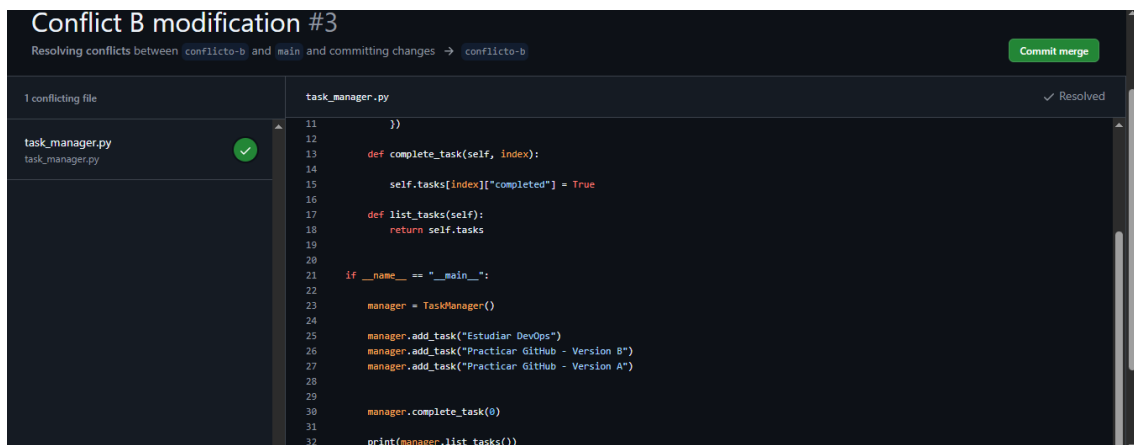
Presionando “Merge conflicts” nos aparece esta pantalla y ahí presionamos “Resolve conflicts”.



Git abrirá un editor propio, y justamente nos muestra el conflicto.

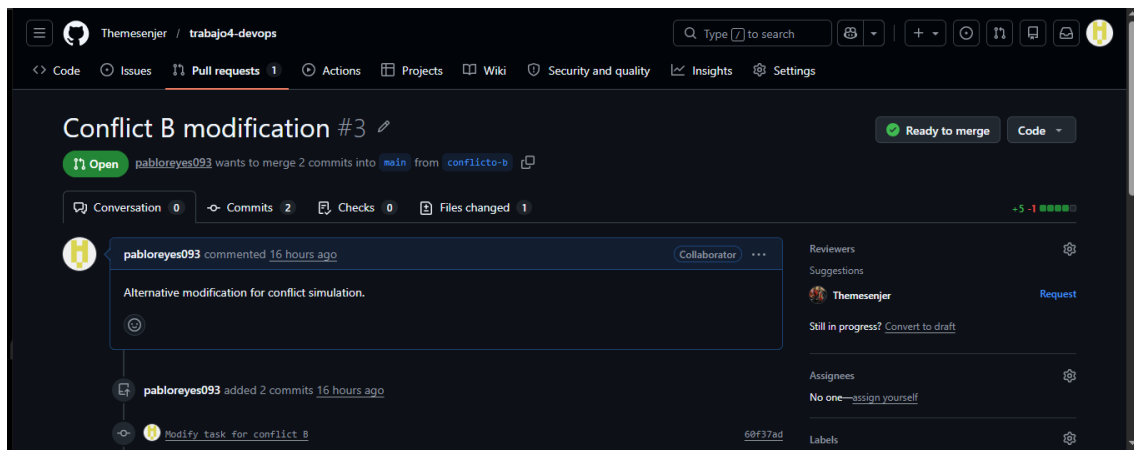


Eliminamos las marcas de conflicto que se mostraban en la imagen anterior, y presionamos “Mark as resolved”.

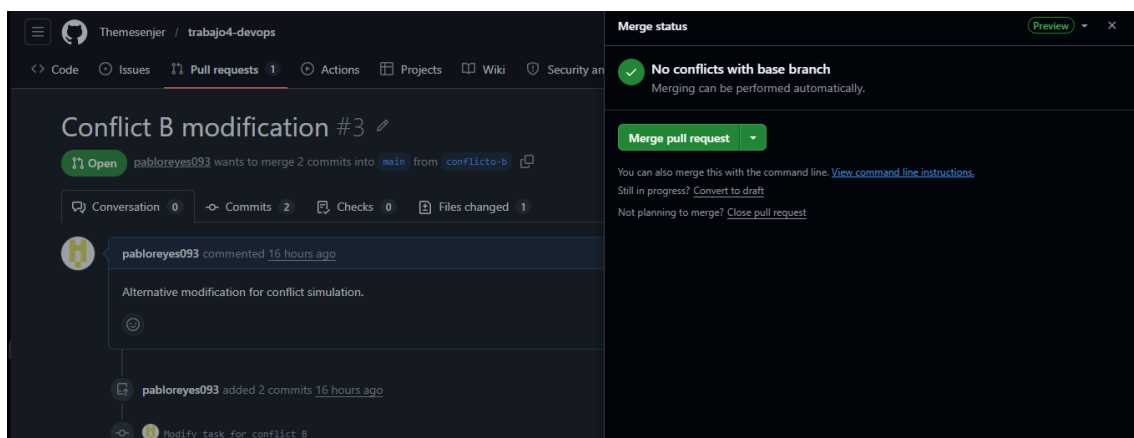


Luego presionamos “Commit Merge”.

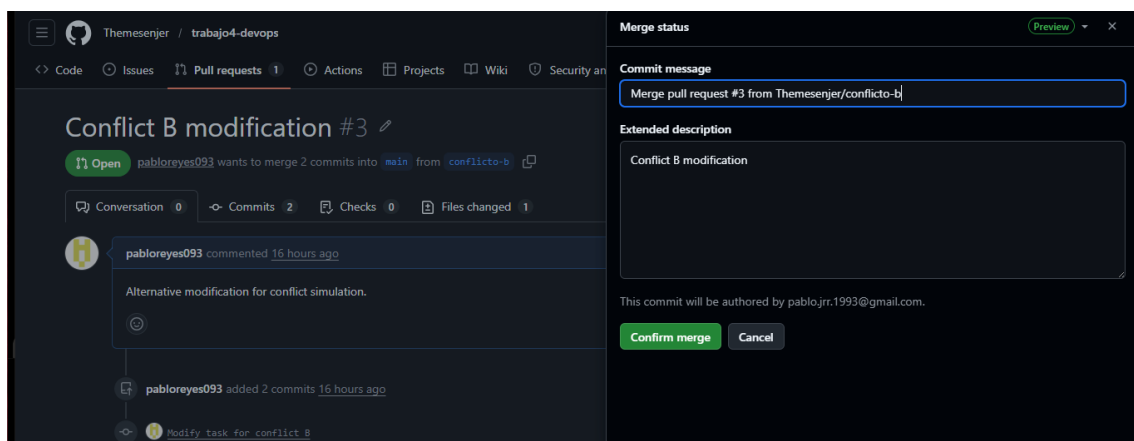
Una vez seguido estos pasos automáticamente Git nos dirige nuevamente a la pagina del PR, en donde haremos el Merge que ya debe estar listo.



Como podemos ver el Merge esta listo.

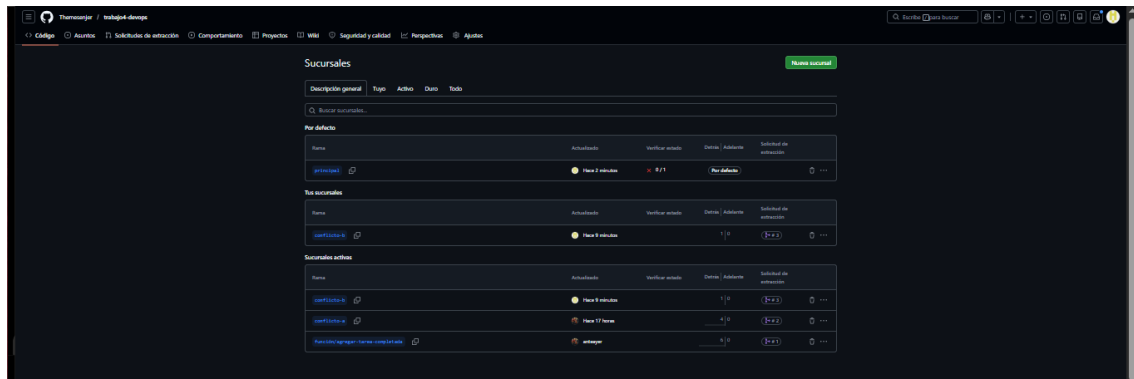


Presionamos “Merge Pull Request”.

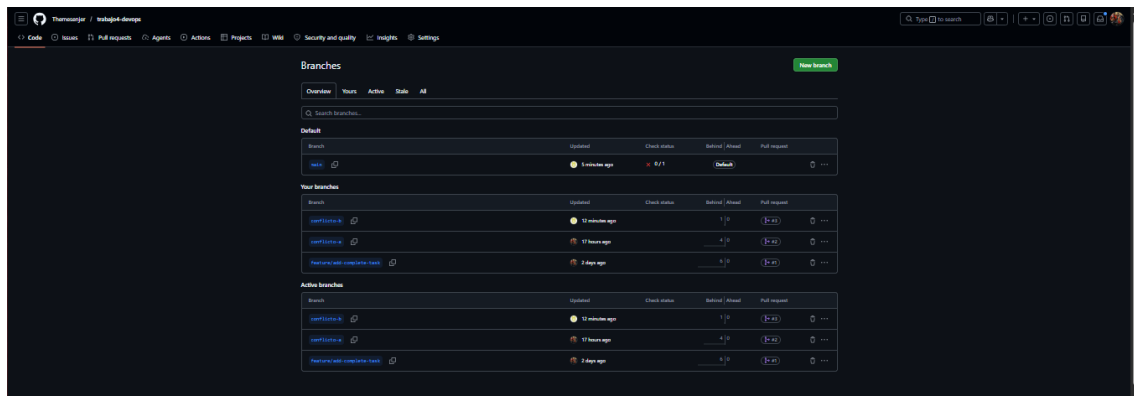


Confirmamos el Merge.

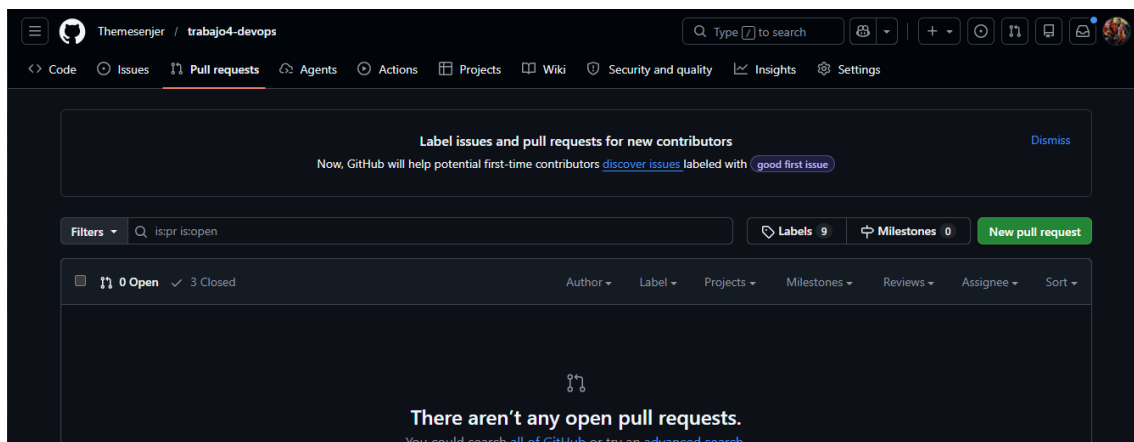
Ahora como paso final veremos todas las ramas que hemos hecho, para la correcta evidencia de su creación, además de eso veremos los PR cerrados y margeados.



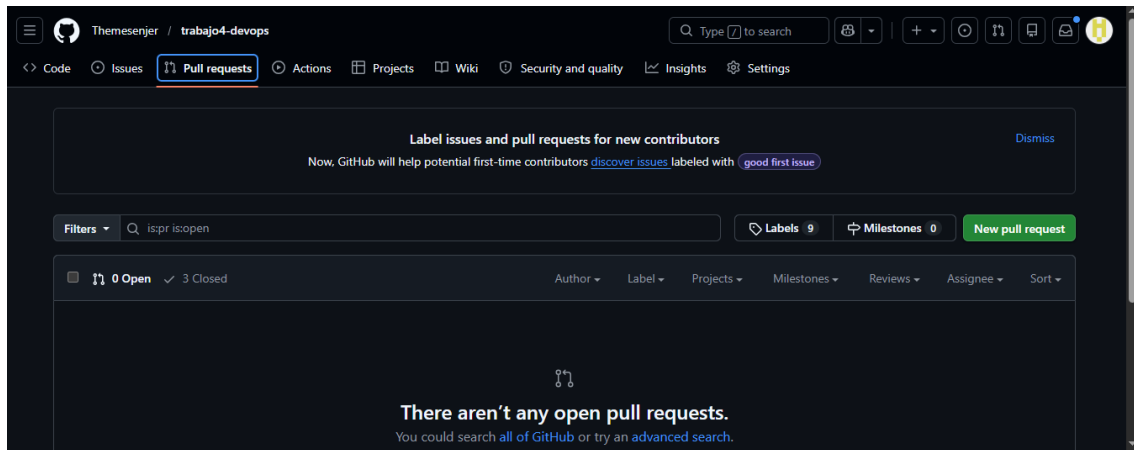
Este es del repo colaborativo donde se observan sus ramas.



Y este es del repo principal donde se observan sus ramas.



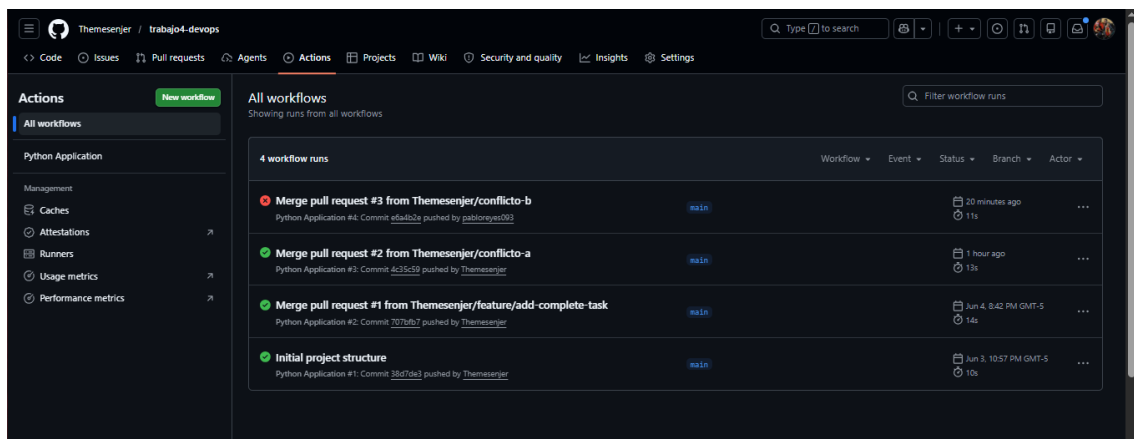
Este es el repo principal donde su PR están cerrados.



Y este es el colaborativo en donde sus PR también están cerrados.

✓ Fase 8 – Automatización e Integración Continua con GitHub Actions

En esta parte final de la practica verificaremos los “Workflows” en Git Actions.



Esos 4 runs nos ayudan mucho porque demuestran la evolución del proyecto.

En otras palabras, observamos esto:

Workflow

Evidencia

Python Application #1 Creación inicial del proyecto

Python Application #2 Integración de nueva funcionalidad (feature/add-complete-task)

Python Application #3 Merge de conflicto-a

Python Application #4 Resolución y merge de conflicto-b realizado por el colaborador

Y además observamos la participación de ambos usuarios de Git, lo cual es importante, porque fortalece la evidencia de trabajo colaborativo entre 2 usuarios, que en un principio fue una de las solicitudes en el trabajo.

✓ Fase 9 – Investigación para poder realizar todo lo hecho hasta ahora.

Esta parte del documento es netamente teórica, la cual cubrirá toda la investigación que se ha hecho para poder realizar la práctica de la tarea.

1. Control de versiones

Definición

El control de versiones es un sistema que permite registrar y gestionar los cambios realizados sobre archivos a lo largo del tiempo. Facilita el seguimiento de modificaciones, la recuperación de versiones anteriores y el trabajo colaborativo entre varios desarrolladores.

Utilidad

Permite mantener un historial de cambios, evitar la pérdida de información y coordinar el trabajo de múltiples personas sobre un mismo proyecto.

Ejemplo práctico

Durante esta práctica se utilizó Git para registrar cada modificación realizada al proyecto mediante commits y ramas de trabajo.

Fuente bibliográfica

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress.

2. Git

Definición

Git es un sistema de control de versiones distribuido diseñado para gestionar proyectos de software de forma eficiente.

Utilidad

Permite registrar cambios, crear ramas de trabajo, fusionar código y colaborar con otros desarrolladores.

Ejemplo práctico

Se utilizó Git para crear ramas como feature/add-complete-task, conflicto-a y conflicto-b.

Fuente bibliográfica

<https://git-scm.com/doc>

3. GitHub

Definición

GitHub es una plataforma basada en la nube que permite alojar repositorios Git y facilitar la colaboración entre desarrolladores.

Utilidad

Permite almacenar proyectos, gestionar Pull Requests, ejecutar GitHub Actions y controlar versiones.

Ejemplo práctico

El proyecto fue alojado en GitHub y se utilizaron Pull Requests para integrar cambios.

Fuente bibliográfica

<https://docs.github.com>

4. Repositorio Local

Definición

Es la copia del repositorio almacenada en el equipo del desarrollador.

Utilidad

Permite trabajar sin conexión y realizar cambios antes de publicarlos.

Ejemplo práctico

El desarrollo del Task Manager se realizó inicialmente en el repositorio local.

Fuente bibliográfica

<https://git-scm.com/book>

5. Repositorio Remoto

Definición

Es una copia del repositorio almacenada en un servidor remoto.

Utilidad

Permite compartir código y sincronizar cambios entre desarrolladores.

Ejemplo práctico

GitHub actuó como repositorio remoto durante la práctica.

Fuente bibliográfica

<https://docs.github.com/en/repositories>

6. Commit

Definición

Un commit representa una confirmación de cambios dentro del historial del repositorio.

Utilidad

Permite guardar versiones específicas del proyecto.

Ejemplo práctico

Se realizaron commits descriptivos para registrar cada avance del desarrollo.

Fuente bibliográfica

<https://git-scm.com/docs/git-commit>

7. Branch (Rama)

Definición

Una rama es una línea independiente de desarrollo dentro de un repositorio Git.

Utilidad

Permite desarrollar funcionalidades sin afectar la rama principal.

Ejemplo práctico

Se crearon ramas para nuevas funcionalidades y para simular conflictos de integración.

Fuente bibliográfica

<https://git-scm.com/docs/git-branch>

8. Merge

Definición

Es el proceso de combinar cambios de una rama en otra.

Utilidad

Permite integrar funcionalidades desarrolladas por distintos colaboradores.

Ejemplo práctico

Las ramas feature/add-complete-task, conflicto-a y conflicto-b fueron integradas mediante merge.

Fuente bibliográfica

<https://git-scm.com/docs/git-merge>

9. Pull Request

Definición

Es una solicitud para revisar e integrar cambios de una rama a otra.

Utilidad

Facilita la revisión de código y la colaboración entre desarrolladores.

Ejemplo práctico

Se crearon Pull Requests para cada funcionalidad implementada.

Fuente bibliográfica

<https://docs.github.com/en/pull-requests>

10. Fork

Definición

Un fork es una copia de un repositorio creada en otra cuenta de GitHub.

Utilidad

Permite experimentar o contribuir a proyectos sin afectar el repositorio original.

Ejemplo práctico

Un desarrollador puede crear un fork de un proyecto de código abierto para proponer mejoras.

Fuente bibliográfica

<https://docs.github.com/en/get-started/quickstart/fork-a-repo>

11. Conflictos de integración

Definición

Ocurren cuando Git no puede determinar automáticamente cómo combinar cambios realizados sobre las mismas líneas de código.

Utilidad

Permiten detectar inconsistencias antes de integrar cambios.

Ejemplo práctico

Durante la práctica se generó un conflicto modificando la misma línea desde las ramas conflicto-a y conflicto-b.

Fuente bibliográfica

<https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts>

12. Integración Continua (CI)

Definición

Es una práctica de desarrollo en la que los cambios se integran frecuentemente y son verificados automáticamente.

Utilidad

Permite detectar errores de forma temprana.

Ejemplo práctico

GitHub Actions ejecutó automáticamente pruebas cada vez que se realizó un push.

Fuente bibliográfica

Martin Fowler. Continuous Integration. <https://martinfowler.com>

13. Entrega Continua (Continuous Delivery)

Definición

Es una práctica que permite que una aplicación esté siempre lista para ser desplegada.

Utilidad

Reduce riesgos durante la publicación de nuevas versiones.

Ejemplo práctico

El pipeline verificó automáticamente el proyecto antes de integrarlo a la rama principal.

Fuente bibliográfica

Humble, J., & Farley, D. Continuous Delivery.

14. Despliegue Continuo (Continuous Deployment)

Definición

Es una extensión de la entrega continua donde los cambios aprobados se despliegan automáticamente a producción.

Utilidad

Permite publicar nuevas versiones sin intervención manual.

Ejemplo práctico

Un sistema puede desplegar automáticamente una nueva versión después de superar todas las pruebas.

Fuente bibliográfica

<https://aws.amazon.com/devops/continuous-deployment/>

15. GitHub Actions

Definición

Es la plataforma de automatización de GitHub para ejecutar workflows de CI/CD.

Utilidad

Permite automatizar pruebas, compilaciones, despliegues y tareas repetitivas.

Ejemplo práctico

Se utilizó un workflow que ejecutó pruebas automáticamente después de cada push y merge realizado durante la práctica.

Fuente bibliográfica

<https://docs.github.com/en/actions>

✓ Fase 10 – Información adicional.

En esta última parte se incluirá datos personales, el link del repositorio, una conclusión personal de lo realizado y la bibliografía completa.

Autor: Pablo Javier Reyes Reyes

Carrera: Desarrollo De Software

Nivel académico: 5to Semestre paralelo “A”

Jornada: Matutina

URL Del Repositorio: <https://github.com/Themesenjer/trabajo4-devops.git>

Conclusión

Durante el desarrollo de esta actividad pude fortalecer mis conocimientos sobre el uso de Git, GitHub y las buenas prácticas utilizadas en entornos de desarrollo de software colaborativo. A través de la creación de ramas, la realización de commits, la generación de Pull Requests, la revisión de código y la ejecución de procesos de merge, comprendí

de manera práctica cómo se gestionan los cambios dentro de un proyecto de software moderno.

Asimismo, la simulación de diferentes roles dentro del equipo de desarrollo me permitió entender tanto las responsabilidades de un desarrollador como las de un revisor de código, logrando experimentar de forma integral el flujo de trabajo utilizado en proyectos profesionales. La generación y resolución de conflictos de integración resultó especialmente valiosa, ya que permitió conocer los problemas más comunes que pueden surgir cuando varios desarrolladores trabajan sobre los mismos archivos.

Adicionalmente, la implementación de GitHub Actions permitió comprender los fundamentos de la Integración Continua (CI), automatizando la validación del proyecto cada vez que se realizaron cambios en el repositorio. Esto evidenció la importancia de la automatización para garantizar la calidad y estabilidad del software.

Finalmente, esta práctica contribuyó significativamente al fortalecimiento de mis competencias en DevOps, control de versiones y trabajo colaborativo, habilidades fundamentales para mi formación profesional como estudiante de Desarrollo de Software y para mi futuro desempeño en proyectos reales dentro de la industria tecnológica.

Bibliografía

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. Recuperado de: <https://git-scm.com/book/en/v2>

Git Documentation. (2026). Git Reference Manual. Recuperado de: <https://git-scm.com/docs>

GitHub Documentation. (2026). GitHub Docs. Recuperado de: <https://docs.github.com>

GitHub Documentation. Pull Requests. Recuperado de: <https://docs.github.com/en/pull-requests>

GitHub Documentation. Fork a Repository. Recuperado de: <https://docs.github.com/en/get-started/quickstart/fork-a-repo>

GitHub Documentation. Managing Merge Conflicts. Recuperado de: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts>

Fowler, M. (2006). Continuous Integration. Recuperado de: <https://martinfowler.com/articles/continuousIntegration.html>

Humble, J., & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley Professional.

Amazon Web Services (AWS). Continuous Deployment. Recuperado de: <https://aws.amazon.com/devops/continuous-deployment/>

GitHub Documentation. GitHub Actions. Recuperado de:
<https://docs.github.com/en/actions>

Atlassian. Git Tutorials. Recuperado de: <https://www.atlassian.com/git/tutorials>

Red Hat. What is DevOps? Recuperado de: <https://www.redhat.com/en/topics/devops>